

Coursera Programming Languages Course

Section 1 Summary

*Standard Description: This summary covers **roughly** the same material as the lecture videos and the materials (slides, code) posted with each video. It can help to read about the material in a narrative style and to have the material for an entire section of the course in a single document, especially when reviewing the material later. Please report errors in these notes on the discussion board.*

Contents

Welcome to Programming Languages	1
ML Expressions and Variable Bindings	2
Using use	3
Variables are Immutable	4
Function Bindings	4
Pairs and Other Tuples	5
Lists	6
Let Expressions	8
Options	10
Some Other Expressions and Operators	11
Lack of Mutation and Benefits Thereof	11
The Pieces of a Programming Language	14

Welcome to Programming Languages

(See also introductory material on the course webpage about course structure, homeworks, grading, software installation, etc. That material is not repeated here.)

A course titled, “Programming Languages” can mean many different things. For us, it means the opportunity to learn the *fundamental concepts* that appear in one form or another in almost every programming language. We will also get some sense of how these concepts “fit together” to provide what programmers need in a language. And we will use different languages to see how they can take complementary approaches to representing these concepts. All of this is intended to make you a better software developer, in any language.

Many people would say this course “teaches” the 3 languages ML (in Part A), Racket (in Part B), and Ruby (Part C), but that is a poor description. We will use these languages to learn various paradigms and concepts because they are well-suited to do so. If our goal were just to make you as productive as possible in these three languages, the course material would be very different. That said, being able to learn new languages and recognize the similarities and differences across languages is an important goal.

Most of the course will use *functional programming* (both ML and Racket are functional languages), which emphasizes immutable data (no assignment statements) and functions, especially functions that take and return other functions. As we will discuss later in Part C, functional programming does some things exactly the opposite of object-oriented programming but also has many similarities. Functional programming is not only a very powerful and elegant approach, but learning it helps you better understand all styles of programming.

The conventional thing to do at the very beginning of a course is to motivate the course, which in this case would explain why you should learn functional programming and more generally why it is worth learning

different languages, paradigms, and language concepts. We will largely *delay* this discussion until after the third homework. It is simply too important to cover when most students are more concerned with getting a sense of what the work in the course will be like, and, more importantly, it is a much easier discussion to have after we have built up shared terminology and experience. Motivation does matter; let's take a "rain-check" with the promise that it will be well worth it.

ML Expressions and Variable Bindings

So let's just start "learning ML" but in a way that teaches core programming-languages concepts rather than just "getting down some code that works." Therefore, pay extremely careful attention to the words used to describe the very, very simple code we start with. We are building a foundation that we will expand very quickly over this week and next week. Do not *yet* try to relate what you see back to what you already know in other languages as that is likely to lead to struggle.

An ML program is a sequence of *bindings*. Each binding gets *type-checked* and then (assuming it type-checks) *evaluated*. What type (if any) a binding has depends on a *static environment*,¹ which is roughly the types of the preceding bindings in the file. How a binding is evaluated depends on a *dynamic environment*, which is roughly the values of the preceding bindings in the file. When we just say *environment*, we usually mean *dynamic environment*. Sometimes *context* is used as a synonym for *static environment*.

There are several kinds of bindings, but for now let's consider only a *variable binding*, which in ML has this *syntax*:

```
val x = e;
```

Here, `val` is a keyword, `x` can be any variable, and `e` can be any *expression*. We will learn many ways to write expressions. The semicolon is optional in a file, but necessary in the *read-eval-print loop* to let the *interpreter* know that you are done typing the binding.

We now know a variable binding's syntax (how to write it), but we still need to know its *semantics* (how it type-checks and evaluates). Mostly this depends on the expression `e`. To type-check a variable binding, we use the "current static environment" (the types of preceding bindings) to type-check `e` (which will depend on what kind of expression it is) and produce a "new static environment" that is the current static environment except with `x` having type `t` where `t` is the type of `e`. Evaluation is analogous: To evaluate a variable binding, we use the "current dynamic environment" (the values of preceding bindings) to evaluate `e` (which will depend on what kind of expression it is) and produce a "new dynamic environment" that is the current environment except with `x` having the value `v` where `v` is the result of evaluating `e`.

A *value* is an expression that, "has no more computation to do," i.e., there is no way to simplify it. As described more generally below, `17` is a value, but `8+9` is not. All values are expressions. Not all expressions are values.

This whole description of what ML programs mean (bindings, expressions, types, values, environments) may seem awfully theoretical or esoteric, but it is exactly the foundation we need to give precise and concise definitions for several different kinds of expressions. Here are several such definitions:

- Integer constants:
 - Syntax: a sequence of digits
 - Type-checking: type `int` in any static environment
 - Evaluation: to itself in any dynamic environment (it is a value)

¹The word *static* here has a tenuous connection to its use in Java/C/C++, but too tenuous to explain at this point.

- Addition:
 - Syntax: `e1+e2` where `e1` and `e2` are expressions
 - Type-checking: type `int` but only if `e1` and `e2` have type `int` in the same static environment, else does not type-check
 - Evaluation: evaluate `e1` to `v1` and `e2` to `v2` in the same dynamic environment and then produce the sum of `v1` and `v2`
- Variables:
 - Syntax: a sequence of letters, underscores, etc.
 - Type-checking: look up the variable in the current static environment and use that type
 - Evaluation: look up the variable in the current dynamic environment and use that value
- Conditionals:
 - Syntax is `if e1 then e2 else e3` where `e1`, `e2`, and `e3` are expressions
 - Type-checking: using the current static environment, a conditional type-checks only if (a) `e1` has type `bool` and (b) `e2` and `e3` have the same type. The type of the whole expression is the type of `e2` and `e3`.
 - Evaluation: under the current dynamic environment, evaluate `e1`. If the result is `true`, the result of evaluating `e2` under the current dynamic environment is the overall result. If the result is `false`, the result of evaluating `e3` under the current dynamic environment is the overall result.
- Boolean constants:
 - Syntax: either `true` or `false`
 - Type-checking: type `bool` in any static environment
 - Evaluation: to itself in any dynamic environment (it is a value)
- Less-than comparison:
 - Syntax: `e1 < e2` where `e1` and `e2` are expressions
 - Type-checking: type `bool` but only if `e1` and `e2` have type `int` in the same static environment, else does not type-check
 - Evaluation: evaluate `e1` to `v1` and `e2` to `v2` in the same dynamic environment and then produce `true` if `v1` is less than `v2` and `false` otherwise

Whenever you learn a new construct in a programming language, you should ask these three questions: What is the syntax? What are the type-checking rules? What are the evaluation rules?

Using use

When using the read-eval-print loop, it is very convenient to add a sequence of bindings from a file.

```
use "foo.sml";
```

does just that. Its type is `unit` and its result is `()` (the only value of type `unit`), but its effect is to include all the bindings in the file `"foo.sml"`.

Variables are Immutable

Bindings are *immutable*. Given `val x = 8+9`; we produce a dynamic environment where `x` maps to 17. In this environment, `x` will *always* map to 17; there is no “assignment statement” in ML for changing what `x` maps to. That is very useful if you are using `x`. You *can* have another binding later, say `val x = 19`;, but that just creates a *different environment* where the later binding for `x` *shadows* the earlier one. This distinction will be extremely important when we define functions that use variables.

Function Bindings

Recall that an ML program is a sequence of bindings. Each binding adds to the static environment (for type-checking subsequent bindings) and to the dynamic environment (for evaluating subsequent bindings). We already introduced variable bindings; we now introduce *function bindings*, i.e., how to define and use functions. We will then learn how to build up and use larger pieces of data from smaller ones using *pairs* and *lists*.

A function is sort of like a method in languages like Java — it is something that is called with arguments and has a body that produces a result. Unlike a method, there is no notion of a class, `this`, etc. We also do not have things like return statements. A simple example is this function that computes x^y assuming $y \geq 0$:

```
fun pow (x:int, y:int) = (* correct only for y >= 0 *)
  if y=0
  then 1
  else x * pow(x,y-1)
```

Syntax:

The syntax for a function binding looks like this (we will generalize this definition a little later in the course):

```
fun x0 (x1 : t1, ..., xn : tn) = e
```

This is a binding for a function named `x0`. It takes n arguments `x1`, ... `xn` of types `t1`, ..., `tn` and has an expression `e` for its body. As always, syntax is just syntax — we must define the typing rules and evaluation rules for function bindings. But roughly speaking, in `e`, the arguments are bound to `x1`, ... `xn` and the result of calling `x0` is the result of evaluating `e`.

Type-checking:

To type-check a function binding, we type-check the body `e` in a static environment that (in addition to all the earlier bindings) maps `x1` to `t1`, ... `xn` to `tn` and `x0` to `t1 * ... * tn -> t`. Because `x0` is in the environment, we can make *recursive* function calls, i.e., a function definition can use itself. The syntax of a function type is “argument types” $\rightarrow$ “result type” where the argument types are separated by `*` (which just happens to be the same character used in expressions for multiplication). For the function binding to type-check, the body `e` must have the type `t`, i.e., the result type of `x0`. That makes sense given the evaluation rules below because the result of a function call is the result of evaluating `e`.

But what, exactly, is `t` – we never wrote it down? It can be any type, and it is up to the type-checker (part of the language implementation) to figure out what `t` should be such that using it for the result type of `x0` makes, “everything work out.” For now, we will take it as magical, but *type inference* (figuring out types not written down) is a very cool feature of ML discussed later in the course. It turns out that in ML you

almost never have to write down types. Soon the argument types $t_1, \dots, t_n$ will also be optional but not until we learn pattern matching a little later.²

After a function binding, x_0 is added to the static environment with its type. The arguments are not added to the top-level static environment — they can be used only in the function body.

Evaluation:

The evaluation rule for a function binding is trivial: *A function is a value* — we simply add x_0 to the environment as a function that can be *called* later. As expected for recursion, x_0 is in the dynamic environment in the function body and for subsequent bindings (but not, unlike in say Java, for preceding bindings, so the order you define functions is very important).

Function calls:

Function bindings are useful only with function calls, a new kind of expression. The *syntax* is $e_0 (e_1, \dots, e_n)$ with the parentheses optional if there is exactly one argument. The *typing rules* require that e_0 has a type that looks like $t_1 * \dots * t_n \rightarrow t$ and for $1 \leq i \leq n$, e_i has type t_i . Then the whole call has type t . Hopefully, this is not too surprising. For the *evaluation rules*, we use the environment at the point of the call to evaluate e_0 to v_0 , e_1 to v_1 , ..., e_n to v_n . Then v_0 must be a function (it will be assuming the call type-checked) and we evaluate the function's body in an environment extended such that the function arguments map to $v_1, \dots, v_n$.

Exactly which environment is it we extend with the arguments? The environment that “was current” when the function was *defined*, not the one where it is being called. This distinction will not arise right now, but we will discuss it in great detail later.

Putting all this together, we can determine that this code will produce an environment where **ans** is 64:

```
fun pow (x:int, y:int) = (* correct only for y >= 0 *)
  if y=0
  then 1
  else x * pow(x,y-1)

fun cube (x:int) =
  pow(x,3)

val ans = cube(4)
```

Pairs and Other Tuples

Programming languages need ways to build compound data out of simpler data. The first way we will learn about in ML is *pairs*. The *syntax* to build a pair is (e_1, e_2) which *evaluates* e_1 to v_1 and e_2 to v_2 and makes the pair of values (v_1, v_2) , which is itself a value. Since v_1 and/or v_2 could themselves be pairs (possibly holding other pairs, etc.), we can build data with several “basic” values, not just two, say, integers. The *type* of a pair is $t_1 * t_2$ where t_1 is the type of the first part and t_2 is the type of the second part.

Just like making functions is useful only if we can call them, making pairs is useful only if we can later retrieve the pieces. Until we learn pattern-matching, we will use **#1** and **#2** to access the first and second part. The typing rule for **#1** e or **#2** e should not be a surprise: e must have some type that looks like $t_a * t_b$ and then **#1** e has type t_a and **#2** e has type t_b .

Here are several example functions using pairs. `div_mod` is perhaps the most interesting because it uses a

²The way we are using pair-reading constructs like **#1** in this lecture and Homework 1 requires these explicit types.

pair to return an answer that has two parts. This is quite pleasant in ML, whereas in Java (for example) returning two integers from a function requires defining a class, writing a constructor, creating a new object, initializing its fields, and writing a return statement.

```
fun swap (pr : int*bool) =
  (#2 pr, #1 pr)

fun sum_two_pairs (pr1 : int*int, pr2 : int*int) =
  (#1 pr1) + (#2 pr1) + (#1 pr2) + (#2 pr2)

fun div_mod (x : int, y : int) = (* note: returning a pair is a real pain in Java *)
  (x div y, x mod y)

fun sort_pair (pr : int*int) =
  if (#1 pr) < (#2 pr)
  then pr
  else ((#2 pr), (#1 pr))
```

In fact, ML supports *tuples* by allowing any number of parts. For example, a 3-tuple (i.e., a triple) of integers has type `int*int*int`. An example is `(7,9,11)` and you retrieve the parts with `#1 e`, `#2 e`, and `#3 e` where `e` is an expression that evaluates to a triple.

Pairs and tuples can be nested however you want. For example, `(7, (true, 9))` is a value of type `int * (bool * int)`, which is different from `((7, true), 9)` which has type `(int * bool) * int` or `(7, true, 9)` which has type `int * bool * int`.

Lists

Though we can nest pairs of pairs (or tuples) as deep as we want, for any variable that has a pair, any function that returns a pair, etc. there has to be a type for a pair and that type will determine the amount of “real data.” Even with tuples the type specifies how many parts it has. That is often too restrictive; we may need a list of data (say integers) and the length of the list is not yet known when we are type-checking (it might depend on a function argument). ML has *lists*, which are more flexible than pairs because they can have any length, but less flexible because all the elements of any particular list must have the same type.

The empty list, with syntax `[]`, has 0 elements. It is a value, so like all values it evaluates to itself immediately. It can have type `t list` for *any* type `t`, which ML writes as `'a list` (pronounced “quote a list” or “alpha list”). In general, the type `t list` describes lists where all the elements in the list have type `t`. That holds for `[]` no matter what `t` is.

A non-empty list with n values is written `[v1,v2,...,vn]`. You can make a list with `[e1,...,en]` where each expression is evaluated to a value. It is more common to make a list with `e1 :: e2`, pronounced “`e1` consed onto `e2`.” Here `e1` evaluates to an “item of type `t`” and `e2` evaluates to a “list of `t` values” and the result is a new list that starts with the result of `e1` and then is all the elements in `e2`.

As with functions and pairs, making lists is useful only if we can then do something with them. As with pairs, we will change how we use lists after we learn pattern-matching, but for now we will use three functions provided by ML. Each takes a list as an argument.

- `null` evaluates to `true` for empty lists and `false` for nonempty lists.
- `hd` returns the first element of a list, *raising an exception* if the list is empty.

- `tl` returns the tail of a list (a list like its argument but without the first element), raising an exception if the list is empty.

Here are some simple examples of functions that take or return lists:

```
fun sum_list (xs : int list) =
  if null xs
  then 0
  else hd(xs) + sum_list(tl xs)

fun countdown (x : int) =
  if x=0
  then []
  else x :: countdown(x-1)

fun append (xs : int list, ys : int list) =
  if null xs
  then ys
  else (hd xs) :: append(tl xs, ys)
```

Functions that make and use lists are almost always recursive because a list has an unknown length. To write a recursive function, the thought process involves thinking about the *base case* — for example, what should the answer be for an empty list — and the *recursive case* — how can the answer be expressed in terms of the answer for the rest of the list.

When you think this way, many problems become much simpler in a way that surprises people who are used to thinking about while loops and assignment statements. A great example is the `append` function above that takes two lists and produces a list that is one list appended to the other. This code implements an elegant recursive algorithm: If the first list is empty, then we can append by just evaluating to the second list. Otherwise, we can append the tail of the first list to the second list. That is almost the right answer, but we need to “cons on” (using `::` has been called “consing” for decades) the first element of the first list. There is nothing magical here — we keep making recursive calls with shorter and shorter first lists and then as the recursive calls complete we add back on the list elements removed for the recursive calls.

Finally, we can combine pairs and lists however we want without having to add any new features to our language. For example, here are several functions that take a list of pairs of integers. Notice how the last function reuses earlier functions to allow for a very short solution. This is very common in functional programming. In fact, it should bother us that `firsts` and `seconds` are so similar but we do not have them share any code. We will learn how to fix that later.

```
fun sum_pair_list (xs : (int * int) list) =
  if null xs
  then 0
  else #1 (hd xs) + #2 (hd xs) + sum_pair_list(tl xs)

fun firsts (xs : (int * int) list) =
  if null xs
  then []
  else (#1 (hd xs))::(firsts(tl xs))

fun seconds (xs : (int * int) list) =
```

```

    if null xs
    then []
    else (#2 (hd xs))::(seconds(tl xs))

fun sum_pair_list2 (xs : (int * int) list) =
    (sum_list (firsts xs)) + (sum_list (seconds xs))

```

Let Expressions

Let-expressions are an absolutely crucial feature that allows for local variables in a very simple, general, and flexible way. Let-expressions are crucial for style and for efficiency. A let-expression lets us have local variables. In fact, it lets us have local *bindings* of any sort, including function bindings. Because it is a kind of expression, it can appear anywhere an expression can.

Syntactically, a let-expression is:

```
let b1 b2 ... bn in e end
```

where each **bi** is a binding and **e** is an expression.

The type-checking and semantics of a let-expression are much like the semantics of the top-level bindings in our ML program. We evaluate each binding in turn, creating a larger environment for the subsequent bindings. So we can use all the earlier bindings for the later ones, and we can use them all for **e**. We call the *scope* of a binding “where it can be used,” so the scope of a binding in a let-expression is the later bindings in that let-expression and the “body” of the let-expression (the **e**). The value **e** evaluates to is the value for the entire let-expression, and, unsurprisingly, the type of **e** is the type for the entire let-expression.

For example, this expression evaluates to 7; notice how one inner binding for **x** *shadows* an outer one.

```

let val x = 1
in
    (let val x = 2 in x+1 end) + (let val y = x+2 in y+1 end)
end

```

Also notice how let-expressions are expressions so they can appear as a subexpression in an addition (though this example is silly and bad style because it is hard to read).

Let-expressions can bind functions too, since functions are just another kind of binding. If a helper function is needed by only one other function and is unlikely to be useful elsewhere, it is good style to bind it locally. For example, here we use a local helper function to help produce the list `[1,2,...,x]`:

```

fun countup_from1 (x:int) =
    let fun count (from:int, to:int) =
            if from=to
            then to::[]
            else from :: count(from+1,to)
        in
            count(1,x)
        end

```

However, we can do better. When we evaluate a call to `count`, we evaluate `count`’s body in a dynamic environment that is the environment where `count` was defined, extended with bindings for `count`’s arguments.

The code above does not really utilize this: `count`'s body uses only `from`, `to`, and `count` (for recursion). It could also use `x`, since that is in the environment when `count` is defined. Then we do not need `to` at all, since in the code above it always has the same value as `x`. So this is better style:

```
fun countup_from1_better (x:int) =
  let fun count (from:int) =
        if from=x
        then x::[]
        else from :: count(from+1)
      in
        count 1
      end
```

This technique — define a local function that uses other variables in scope — is a hugely common and convenient thing to do in functional programming. It is a shame that many non-functional languages have little or no support for doing something like it.

Local variables are often good style for keeping code readable. They can be much more important than that when they bind to the *results of* potentially expensive computations. For example, consider this code that does not use let-expressions:

```
fun bad_max (xs : int list) =
  if null xs
  then 0 (* note: bad style; see below *)
  else if null (tl xs)
  then hd xs
  else if hd xs > bad_max(tl xs)
  then hd xs
  else bad_max(tl xs)
```

If you call `bad_max` with `countup_from1 30`, it will make approximately 2^{30} (over one billion) recursive calls to itself. The reason is an “exponential blowup” — the code calls `bad_max(tl xs)` twice and each of those calls call `bad_max` two more times (so four total) and so on. This sort of programming “error” can be difficult to detect because it can depend on your test data (if the list counts down, the algorithm makes only 30 recursive calls instead of 2^{30}).

We can use let-expressions to avoid repeated computations. This version computes the max of the tail of the list once and stores the resulting value in `tl_ans`.

```
fun good_max (xs : int list) =
  if null xs
  then 0 (* note: bad style; see below *)
  else if null (tl xs)
  then hd xs
  else
    (* for style, could also use a let-binding for hd xs *)
    let val tl_ans = good_max(tl xs)
    in
      if hd xs > tl_ans
      then hd xs
      else tl_ans
    end
```

Options

The previous example does not properly handle the empty list — it returns 0. This is bad style because 0 is really not the maximum value of 0 numbers. There is no good answer, but we should deal with this case reasonably. One possibility is to raise an exception; you can learn about ML exceptions on your own if you are interested before we discuss them later in the course. Instead, let's change the return type to either return the maximum number or indicate the input list was empty so there is no maximum. Given the constructs we have, we could “code this up” by return an `int list`, using `[]` if the input was the empty list and a list with one integer (the maximum) if the input list was not empty.

While that works, lists are “overkill” — we will always return a list with 0 or 1 elements. So a list is not really a precise description of what we are returning. The ML library has “options” which are a precise description: an option value has either 0 or 1 thing: `NONE` is an option value “carrying nothing” whereas `SOME e` evaluates `e` to a value `v` and becomes the option carrying the one value `v`. The type of `NONE` is `'a option` and the type of `SOME e` is `t option` if `e` has type `t`.

Given a value, how do you use it? Just like we have `null` to see if a list is empty, we have `isSome` which evaluates to `false` if its argument is `NONE`. Just like we have `hd` and `tl` to get parts of lists (raising an exception for the empty list), we have `valOf` to get the value carried by `SOME` (raising an exception for `NONE`).

Using options, here is a better version with return type `int option`:

```
fun better_max (xs : int list) =
  if null xs
  then NONE
  else
    let val tl_ans = better_max(tl xs)
    in if isSome tl_ans andalso valOf tl_ans > hd xs
       then tl_ans
       else SOME (hd xs)
    end
```

The version above works just fine and is a reasonable recursive function because it does not repeat any potentially expensive computations. But it is both awkward and a little inefficient to have each recursive call except the last one create an option with `SOME` just to have its caller access the value underneath. Here is an alternative approach where we use a local helper function for non-empty lists and then just have the outer function return an option. Notice the helper function would raise an exception if called with `[]`, but since it is defined locally, we can be sure that will never happen.

```
fun better_max2 (xs : int list) =
  if null xs
  then NONE
  else let (* fine to assume argument nonempty because it is local *)
        fun max_nonempty (xs : int list) =
          if null (tl xs) (* xs must not be [] *)
          then hd xs
          else let val tl_ans = max_nonempty(tl xs)
                in
                  if hd xs > tl_ans
                  then hd xs
                  else tl_ans
                end
        in max_nonempty xs
      end
```

```

in
    SOME (max_nonempty xs)
end

```

Some Other Expressions and Operators

ML has all the arithmetic and logical operators you need, but the syntax is sometimes different than in most languages. Here is a brief list of some additional forms of expressions we will find useful:

- `e1 andalso e2` is logical-and: It evaluates `e2` only if `e1` evaluates to `true`. The result is `true` if `e1` and `e2` evaluate to true. Naturally, `e1` and `e2` must both have type `bool` and the entire expression also has type `bool`. In many languages, such expressions are written `e1 && e2`, but that is not the ML syntax, nor is `e1 and e2` (but `and` is a keyword we will encounter later for a different purpose). Using `e1 andalso e2` is generally better style than the equivalent `if e1 then e2 else false`.
- `e1 orelse e2` is logical-or: It evaluates `e2` only if `e1` evaluates to `false`. The result is `true` if `e1` or `e2` evaluates to true. Naturally, `e1` and `e2` must both have type `bool` and the entire expression also has type `bool`. In many languages, such expressions are written `e1 || e2`, but that is not the ML syntax, nor is `e1 or e2`. Using `e1 orelse e2` is generally better style than the equivalent `if e1 then true else e2`.
- `not e` is logical-negation. `not` is just a provided function of type `bool->bool` that we could have defined ourselves as `fun not x = if x then false else true`. In many languages, such expressions are written `!e`, but in ML the `!` operator means something else (related to mutable variables, which we will not use).
- You can compare many values, including integers, for equality using `e1 = e2`.
- Instead of writing `not (e1 = e2)` to see if two numbers are different, better style is `e1 <> e2`. In many languages, the syntax is `e1 != e2`, whereas ML's `<>` can be remembered as, “less than or greater than.”
- The other arithmetic comparisons have the same syntax as in most languages: `>`, `<`, `>=`, `<=`.
- Subtraction is written `e1 - e2`, but it must take two operands, so you *cannot* just write `- e` for negation. For negation, the correct syntax is `~ e`, in particular negative numbers are written like `~7`, *not* `-7`. Using `~e` is better style than `0 - e`, but equivalent for integers.

Lack of Mutation and Benefits Thereof

In ML, there is no way to *change* the contents of a binding, a tuple, or a list. If `x` maps to some value like the list of pairs `[(3,4),(7,9)]` in some environment, then `x` will forever map to that list in that environment. There is no assignment statement that changes `x` to map to a different list. (You can introduce a new binding that shadows `x`, but that will not affect any code that looks up the “original” `x` in an environment.) There is no assignment statement that lets you change the head or tail of a list. And there is no assignment statement that lets you change the contents of a tuple. So we have constructs for building compound data and accessing the pieces, but no constructs for *mutating* the data we have built.

This is a really powerful feature! That may surprise you: how can a language *not* having something be a feature? Because if there is no such feature, then when you are writing *your code* you can rely on *no other code* doing something that would make your code wrong, incomplete, or difficult to use. Having

immutable data is probably the most important “non-feature” a language can have, and it is one of the main contributions of functional programming.

While there are various advantages to immutable data, here we will focus on a big one: it makes sharing and aliasing irrelevant. Let’s re-consider two examples from above before picking on Java (and every other language where mutable data is the norm and assignment statements run rampant).

```
fun sort_pair (pr : int*int) =  
  if (#1 pr) < (#2 pr)  
  then pr  
  else ((#2 pr), (#1 pr))
```

In `sort_pair`, we clearly build and return a new pair in the else-branch, but in the then-branch, do we return a *copy* of the pair referred to by `pr` or do we return an *alias*, where a caller like:

```
val x = (3,4)  
val y = sort_pair x
```

would now have `x` and `y` be aliases for the *same* pair? The answer is *you cannot tell* — there is no construct in ML that can figure out whether or not `x` and `y` are aliases, and no reason to worry that they might be. *If* we had mutation, life would be different. Suppose we could say, “change the second part of the pair `x` is bound to so that it holds 5 instead of 4.” Then we would have to wonder if `#2 y` would be 4 or 5.

In case you are curious, we would expect that the code above would create aliasing: by returning `pr`, the `sort_pair` function would return an alias to its argument. That is more efficient than this version, which would create another pair with exactly the same contents:

```
fun sort_pair (pr : int*int) =  
  if (#1 pr) < (#2 pr)  
  then (#1 pr, #2 pr)  
  else ((#2 pr), (#1 pr))
```

Making the new pair `(#1 pr, #2 pr)` is bad style, since `pr` is simpler and will do just as well. Yet in languages with mutation, programmers make copies like this all the time, exactly to prevent aliasing where doing an assignment using one variable like `x` causes unexpected changes to using another variable like `y`. In ML, no users of `sort_pair` can ever tell whether we return a new pair or not.

Our second example is our elegant function for list append:

```
fun append (xs : int list, ys : int list) =  
  if null xs  
  then ys  
  else (hd xs) :: append(tl xs, ys)
```

We can ask a similar question: Does the list returned *share* any elements with the arguments? Again the answer does not matter because no caller can tell. And again the answer happens to be yes: we build a new list that “reuses” all the elements of `ys`. This saves space, but would be very confusing if someone could later mutate `ys`. Saving space is a nice advantage of immutable data, but so is simply not having to worry about whether things are aliased or not when writing down elegant algorithms.

In fact, `tl` itself thankfully introduces aliasing (though you cannot tell): it returns (an alias to) the tail of the list, which is always “cheap,” rather than making a copy of the tail of the list, which is “expensive” for long lists.

The `append` example is very similar to the `sort_pair` example, but it is even more compelling because it is hard to keep track of potential aliasing if you have many lists of potentially large lengths. If I append `[1,2]` to `[3,4,5]`, I will get *some* list `[1,2,3,4,5]` but if later someone can *change* the `[3,4,5]` list to be `[3,7,5]` is the appended list still `[1,2,3,4,5]` or is it now `[1,2,3,7,5]`?

In the analogous Java program, this is a crucial question, which is why Java programmers *must obsess* over when references to old objects are used and when new objects are created. There are times when obsessing over aliasing is the right thing to do and times when avoiding mutation is the right thing to do — functional programming will help you get better at the latter.

For a final example, the following Java is the key idea behind an actual security hole in an important (and subsequently fixed) Java library. Suppose we are maintaining permissions for who is allowed to access something like a file on the disk. It is fine to let everyone see *who has permission*, but clearly only those that do have permission can actually use the resource. Consider this wrong code (some parts omitted if not relevant):

```
class ProtectedResource {
    private Resource theResource = ...;
    private String[] allowedUsers = ...;
    public String[] getAllowedUsers() {
        return allowedUsers;
    }
    public String currentUser() { ... }
    public void useTheResource() {
        for(int i=0; i < allowedUsers.length; i++) {
            if(currentUser().equals(allowedUsers[i])) {
                ... // access allowed: use it
                return;
            }
        }
        throw new IllegalAccessException();
    }
}
```

Can you find the problem? Here it is: `getAllowedUsers` returns an alias to the `allowedUsers` array, so any user can gain access by doing `getAllowedUsers()[0] = currentUser()`. Oops! This would not be possible if we had some sort of array in Java that did not allow its contents to be updated. Instead, in Java we often have to remember to *make a copy*. The correction below shows an explicit loop to show in detail what must be done, but better style would be to use a library method like `System.arraycopy` or similar methods in the `Arrays` class — these library methods exist because array copying is necessarily common, in part due to mutation.

```
public String[] getAllowedUsers() {
    String[] copy = new String[allowedUsers.length];
    for(int i=0; i < allowedUsers.length; i++)
        copy[i] = allowedUsers[i];
    return copy;
}
```

The Pieces of a Programming Language

Now that we have learned enough ML to write some simple functions and programs with it, we can list the essential “pieces” necessary for defining and learning *any* programming language:

- Syntax: How do you write the various parts of the language?
- Semantics: What do the various language features mean? For example, how are expressions evaluated?
- Idioms: What are the common approaches to using the language features to express computations?
- Libraries: What has already been written for you? How do you do things you could not do without library support (like access files)?
- Tools: What is available for manipulating programs in the language (compilers, read-eval-print loops, debuggers, ...)

While libraries and tools are essential for being an effective programmer (to avoid reinventing available solutions or unnecessarily doing things manually), this course does not focus on them much. That can leave the wrong impression that we are using “silly” or “impractical” languages, but libraries and tools are just less relevant in a course on the conceptual similarities and differences of programming languages.